

SKRIPSI

**ANALISIS EFEKTIVITAS ROLE-BASED MULTI-AGENT LLM DALAM
DETEKSI AMBIGUITAS REQUIREMENT PERANGKAT LUNAK**

***AN EMPIRICAL EVALUATION OF ROLE-BASED MULTI-AGENT LLMS
FOR SOFTWARE REQUIREMENT AMBIGUITY DETECTION***

Usulan Penelitian



ANDREANDHIKI RIYANTA PUTRA
23/517511/PA/22191

**PROGRAM STUDI ILMU KOMPUTER
DEPERTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

CATATAN DRAFT PENELITIAN

Dokumen ini merupakan draft kerja untuk diskusi bimbingan. Fokus, dataset, konfigurasi model, dan jadwal di dalamnya masih dapat disesuaikan setelah validasi literatur, ketersediaan data, dan arahan pembimbing.

Daftar Isi

Halaman Judul	i
Catatan Draft Penelitian	ii
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Perumusan Masalah	2
1.3 Batasan Penelitian	3
1.4 Tujuan Penelitian	3
1.5 Manfaat dan Kontribusi yang Diharapkan	4
II PENELITIAN TERKAIT	5
2.1 Konsep Dasar	5
2.1.1 Requirement dan Ambiguitas	5
2.1.2 Single-Agent LLM	5
2.1.3 Multi-Agent dan Mixture-of-Agents	5
2.1.4 Self-MoA sebagai Pembanding	7
2.2 Studi Terkait	7
2.2.1 Deteksi Ambiguitas Menggunakan LLM	7
2.2.2 Multi-Agent untuk Requirements Engineering	7
2.2.3 Penerapan Multi-Agent pada Domain Ambigu	8
2.3 Sintesis dan Kesenjangan Penelitian	8
2.4 Hipotesis Penelitian	8
III METODE DAN RANCANGAN PENELITIAN	10
3.1 Rancangan Penelitian	10
3.1.1 Tahap 1: Definisi Task dan Preparasi Dataset	10
3.1.2 Tahap 2: Pedoman Anotasi dan Validasi Label	10
3.1.3 Tahap 3: Arsitektur yang Dibandingkan	11
3.1.4 Tahap 4: Protokol Eksperimen	12
3.1.5 Tahap 5: Metrik Evaluasi	13
3.1.6 Tahap 6: Ablation Study dan Analisis Failure Mode	14
3.2 Ancaman terhadap Validitas	14
3.3 Alat dan Bahan	15
3.4 Keluaran Penelitian	15
IV JADWAL PENELITIAN	16
4.1 Rencana Kerja	16
GLOSARIUM	17

Bab I

PENDAHULUAN

1.1 Latar Belakang

Requirement perangkat lunak menjadi media utama untuk menyepakati kebutuhan bisnis, perilaku sistem, dan kriteria penerimaan sebelum pekerjaan desain serta implementasi dilakukan. Apabila satu requirement dapat ditafsirkan dengan cara yang berbeda oleh stakeholder, pengembang, dan penguji, perbedaan tersebut dapat menyebar ke tahap berikutnya dan menimbulkan rework. Ambiguitas dalam requirement dapat muncul pada pilihan kata, struktur kalimat, makna yang tidak spesifik, informasi yang belum lengkap, maupun ketidakkonsistenan antarpernyataan Berry et al. (2000); Kamsties et al. (1998); Wiegers (2003). Literatur Requirements Engineering (RE) umumnya membahas ambiguitas leksikal, sintaktis, semantik, kekaburan (*vagueness*), ketidaklengkapan (*incompleteness*), dan kontradiksi sebagai kategori yang saling melengkapi Berry and Kamsties (2004).

Deteksi ambiguitas secara manual masih bergantung pada pengalaman dan perspektif pembaca. Pendekatan berbasis aturan dan Natural Language Inference (NLI) telah digunakan untuk membantu proses tersebut Ferrara et al. (2014), tetapi aturan yang kaku dapat kesulitan menangkap konteks dan variasi bahasa. Large Language Model (LLM) kemudian digunakan sebagai *single-agent* untuk mengidentifikasi ambiguitas sekaligus menjelaskan alasan deteksinya. Studi industri Bashir et al. (2025) menunjukkan potensi LLM untuk tugas ini melalui konfigurasi prompting yang menggunakan contoh dan keluaran penjelasan Bashir et al. (2025). Namun, satu model yang menghasilkan satu analisis tetap dapat melewatkan ambiguitas yang hanya terlihat dari sudut pandang tertentu.

Perspektif yang berbeda penting karena pihak yang berkepentingan membaca requirement dengan tujuan yang berbeda. Stakeholder lebih sensitif terhadap kebutuhan bisnis yang samar atau belum lengkap. Developer lebih sensitif terhadap istilah, struktur, dan perilaku yang sulit diimplementasikan. Tester lebih sensitif terhadap kriteria penerimaan yang tidak dapat diverifikasi, kasus batas, dan konflik antarpernyataan. Perbedaan ini memberikan alasan konseptual untuk menguji arsitektur yang memberi instruksi dan tanggung jawab khusus kepada beberapa agen, bukan hanya mengulang prompt yang sama.

Paradigma multi-agent dan Mixture-of-Agents (MoA) mengorkestrasi beberapa keluaran model untuk membentuk keputusan akhir Wang et al. (2024). Multi-Agent Debate juga menunjukkan bahwa interaksi antar-agen dapat membantu penalaran pada tugas tertentu Du et al. (2023). Di dalam RE, Oriol et al. (2025) mengevaluasi strategi debat multi-agen untuk klasifikasi requirement, sedangkan Jin et al. (2025) mengusulkan beberapa peran untuk mendukung siklus RE yang lebih luas Oriol et al. (2025); Jin et al. (2025). Penelitian-penelitian tersebut menunjukkan kelayakan orkestrasi multi-agent, tetapi belum secara khusus menjawab apakah spesialisasi

peran meningkatkan deteksi ambiguitas requirement.

Perbandingan dengan *Self-MoA* diperlukan agar manfaat role-based tidak disamakan dengan manfaat dari penambahan jumlah panggilan model. Pada *Self-MoA*, model yang sama dapat diminta menghasilkan beberapa analisis independen lalu digabungkan. Li et al. (2025) menunjukkan bahwa pengulangan model yang sama dapat menjadi konfigurasi yang kuat, sehingga keunggulan arsitektur role-based harus dibuktikan melalui eksperimen yang mengontrol model, prompt budget, dan data uji Li et al. (2025). Tanpa pembanding tersebut, peningkatan hasil belum dapat dikaitkan dengan spesialisasi peran.

Dimensi bahasa juga perlu dipertimbangkan. Sebagian besar kajian deteksi ambiguitas yang menjadi rujukan awal menggunakan bahasa Inggris, sedangkan requirement berbahasa Indonesia memiliki pilihan kata, struktur, dan konteks penggunaan yang berbeda. Karena itu, evaluasi pada bahasa Indonesia dan Inggris dapat menunjukkan apakah arsitektur yang sama tetap konsisten lintas bahasa atau membutuhkan penyesuaian prompt dan data. Studi ini tidak menganggap bahasa Indonesia sebagai sekadar terjemahan, tetapi mencatat proses pembentukan dan validasi dataset agar perbedaan hasil dapat ditafsirkan dengan hati-hati.

Berdasarkan telaah awal tersebut, draft penelitian ini mengusulkan evaluasi empiris terhadap tiga konfigurasi: (1) single-agent LLM, (2) *Self-MoA*, dan (3) role-based multi-agent LLM dengan agen stakeholder, developer, dan tester. Ketiganya diuji pada unit requirement yang sama, menggunakan protokol prompting dan anggaran keluaran yang dapat dibandingkan. Evaluasi mencakup performa keseluruhan, performa setiap kategori ambiguitas, perbedaan lintas bahasa, kontribusi tiap peran, serta waktu dan konsumsi token.

1.2 Perumusan Masalah

Berdasarkan latar belakang tersebut, perumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang arsitektur role-based multi-agent LLM untuk memadukan perspektif stakeholder, developer, dan tester dalam deteksi ambiguitas requirement?
2. Sejauh mana performa role-based multi-agent LLM berbeda dari single-agent LLM dan *Self-MoA* ketika model, data, dan anggaran keluaran dikendalikan?
3. Peran dan kategori ambiguitas apa yang paling berkontribusi terhadap perubahan performa sistem?
4. Bagaimana bahasa requirement (Indonesia atau Inggris) memengaruhi performa deteksi dan biaya inferensi pada setiap konfigurasi?

1.3 Batasan Penelitian

Untuk menjaga penelitian tetap terukur dan dapat diselesaikan, batasan penelitian ditetapkan sebagai berikut:

1. Unit analisis dibatasi pada requirement perangkat lunak berbentuk teks naratif dalam bahasa Indonesia dan Inggris. Model formal, diagram, dan requirement yang memerlukan akses ke sistem privat tidak termasuk dalam objek utama.
2. Pelabelan menggunakan skema multi-label dengan enam kategori: leksikal, sintaktis, semantik, kekaburan, ketidaklengkapan, dan kontradiksi. Satu requirement dapat memiliki lebih dari satu label.
3. Arsitektur yang diusulkan menggunakan tiga peran analisis: stakeholder, developer, dan tester. Satu komponen aggregator digunakan untuk menggabungkan keluaran, tetapi tidak diperlakukan sebagai perspektif tambahan dalam analisis kontribusi peran.
4. Konfigurasi pembandingan terdiri atas single-agent LLM, Self-MoA dengan beberapa instans model yang sama, dan role-based multi-agent LLM. Perbandingan utama menggunakan backbone model yang sama agar pengaruh orkestrasi dapat dipisahkan dari pengaruh kualitas model.
5. Dataset dibagi menjadi bagian pengembangan, validasi, dan pengujian. Data pengujian tidak digunakan untuk memilih prompt, jumlah agen, atau aturan agregasi.
6. Pemilihan model dan parameter akhir menyesuaikan ketersediaan infrastruktur. Model dan versinya akan dicatat agar eksperimen dapat direplikasi; klaim penelitian tidak bergantung pada satu penyedia layanan.
7. Penelitian mengukur kemampuan deteksi dan penjelasan berbasis teks, bukan pengukuran langsung pengurangan defect pada proyek perangkat lunak yang sedang berjalan.

1.4 Tujuan Penelitian

Tujuan penelitian ini adalah:

1. Merancang prototipe role-based multi-agent LLM untuk deteksi ambiguitas requirement perangkat lunak.
2. Membandingkan performa single-agent LLM, Self-MoA, dan role-based multi-agent LLM menggunakan metrik precision, recall, dan F1-score.
3. Menganalisis kontribusi peran stakeholder, developer, dan tester terhadap deteksi kategori ambiguitas tertentu.

4. Menganalisis perbedaan performa dan biaya inferensi pada requirement berbahasa Indonesia dan Inggris.
5. Mendokumentasikan failure modes, proses anotasi, serta kondisi eksperimen agar hasil penelitian dapat diaudit dan direplikasi.

1.5 Manfaat dan Kontribusi yang Diharapkan

Hasil penelitian diharapkan memberikan kontribusi sebagai berikut:

1. **Kontribusi metodologis:** Protokol eksperimen untuk membedakan dampak jumlah agen dari dampak spesialisasi peran pada deteksi ambiguitas requirement.
2. **Kontribusi empiris:** Bukti perbandingan antara single-agent, Self-MoA, dan role-based multi-agent pada task, kategori, dan bahasa yang sama.
3. **Kontribusi praktis:** Prototipe dan skema keluaran yang dapat membantu penulis requirement menemukan bagian yang perlu diklarifikasi sebelum implementasi.
4. **Kontribusi lokal:** Analisis awal terhadap requirement berbahasa Indonesia yang dilengkapi dokumentasi proses validasi, sehingga keterbatasan data dan bahasa dapat dilaporkan secara transparan.

Bab II

PENELITIAN TERKAIT

2.1 Konsep Dasar

2.1.1 Requirement dan Ambiguitas

Requirement adalah pernyataan mengenai kebutuhan, layanan, batasan, atau perilaku yang diharapkan dari perangkat lunak. Requirement yang baik perlu dapat dipahami secara konsisten, ditelusuri, dan diverifikasi. Ambiguitas terjadi ketika satu pernyataan memungkinkan dua atau lebih interpretasi yang masuk akal. Ambiguitas tidak selalu disebabkan oleh satu kata; hubungan dengan requirement lain dan konteks penggunaan juga dapat mengubah interpretasi Berry et al. (2000); Kamsties et al. (1998).

Draft penelitian ini menggunakan taksonomi kerja yang diringkas pada Tabel 2.1. Kategori tersebut digunakan sebagai label analisis, bukan sebagai klaim bahwa semua jenis ambiguitas dapat dipisahkan secara sempurna. Pedoman anotasi akan memuat contoh positif, contoh negatif, dan aturan untuk kasus yang memiliki lebih dari satu label.

2.1.2 Single-Agent LLM

Pada konfigurasi single-agent, satu model menerima requirement dan instruksi analisis, kemudian mengeluarkan label, kategori, bukti, dan penjelasan. Pendekatan ini sederhana serta mudah digunakan sebagai baseline, tetapi semua keputusan bergantung pada satu jalur penalaran dan satu konfigurasi prompt. Pendekatan berbasis NLI telah lebih dahulu digunakan untuk mendukung deteksi ambiguitas Ferrara et al. (2014). Studi industri Bashir et al. (2025) memperlihatkan bahwa LLM dapat digunakan untuk mendeteksi dan menjelaskan ambiguitas requirement, sehingga menjadi rujukan langsung bagi baseline penelitian ini Bashir et al. (2025).

2.1.3 Multi-Agent dan Mixture-of-Agents

MoA menggabungkan keluaran beberapa agen atau model melalui satu atau lebih tahap agregasi Wang et al. (2024). Setiap agen dapat menghasilkan kandidat analisis secara paralel, sedangkan aggregator menyusun keputusan akhir. Multi-Agent Debate menambahkan interaksi atau putaran kritik antar-agen untuk memperbaiki jawaban Du et al. (2023). Dua pola ini berbeda dari sekadar menjalankan single-agent berulang karena keluaran satu agen dapat menjadi konteks bagi agen lain.

Role-based multi-agent memberikan instruksi, tujuan, dan kriteria perhatian yang berbeda kepada setiap agen. Dalam penelitian ini, perbedaan tersebut diwujudkan melalui tiga perspektif: stakeholder, developer, dan tester. Hipotesisnya bukan bahwa satu peran selalu lebih benar, melainkan bahwa kombinasi perspektif dapat

Tabel 2.1: Taksonomi ambiguitas requirement yang digunakan sebagai skema kerja

Kategori	Pengertian operasional	Contoh indikasi
Leksikal	Satu istilah atau frasa memiliki lebih dari satu makna yang relevan.	Istilah “cepat” digunakan tanpa definisi batas waktu.
Sintaktis	Struktur kalimat memungkinkan lebih dari satu cara pengelompokan atau hubungan antarsur.	Keterangan waktu atau objek dapat melekat pada lebih dari satu bagian kalimat.
Semantik	Makna pernyataan tidak dapat dipastikan dari konteks yang tersedia.	Kata “sesuai” atau “normal” tidak memiliki kriteria rujukan.
Kekaburan	Requirement menggunakan ukuran atau batas yang tidak cukup spesifik.	“Sistem harus merespons dengan baik” tanpa ambang yang dapat diuji.
Ketidaklengkapan	Informasi yang diperlukan untuk memahami atau memverifikasi requirement belum tersedia.	Aktor, kondisi awal, atau hasil yang diharapkan tidak disebutkan.
Kontradiksi	Pernyataan bertentangan dengan dirinya sendiri atau dengan requirement lain.	Dua requirement menetapkan perilaku berbeda untuk kondisi yang sama.

meningkatkan cakupan temuan tanpa mengorbankan terlalu banyak biaya inferensi.

2.1.4 Self-MoA sebagai Pembanding

Self-MoA menggunakan model yang sama pada beberapa jalur analisis, lalu menggabungkan hasilnya. Konfigurasi ini penting karena peningkatan performa multi-agent dapat berasal dari variasi keluaran, jumlah sampel, atau agregasi, bukan dari label peran. Li et al. (2025) menekankan bahwa penggunaan model yang sama dapat menjadi pembanding yang kuat terhadap konfigurasi dengan model atau instruksi berbeda Li et al. (2025). Oleh sebab itu, eksperimen utama menyamakan backbone, jumlah panggilan analisis, dan format keluaran antara Self-MoA dan role-based multi-agent.

2.2 Studi Terkait

2.2.1 Deteksi Ambiguitas Menggunakan LLM

Bashir et al. (2025) mengevaluasi deteksi dan penjelasan ambiguitas requirement menggunakan LLM dalam konteks industri Bashir et al. (2025). Relevansinya terhadap penelitian ini terletak pada penggunaan LLM sebagai pemeriksa ambiguitas dan pada kebutuhan untuk menyertakan alasan yang dapat ditinjau manusia. Perbedaannya, penelitian ini menambahkan beberapa perspektif agen dan menguji kontribusi arsitektur tersebut secara terkontrol.

Vijayvargiya et al. (2025) mengusulkan Ambig-SWE, yaitu agen interaktif untuk menghadapi ambiguitas pada tugas software engineering Vijayvargiya et al. (2025). Fokusnya adalah penyelesaian tugas dan interaksi agen, bukan klasifikasi kategori ambiguitas pada requirement. Penelitian tersebut tetap relevan karena menunjukkan bahwa ambiguitas dapat diperlakukan sebagai objek analisis dan umpan balik dalam alur kerja agen.

2.2.2 Multi-Agent untuk Requirements Engineering

Oriol et al. (2025) menerapkan Multi-Agent Debate pada RE dan melaporkan manfaat interaksi antar-agen untuk klasifikasi requirement Oriol et al. (2025). Pendekatan tersebut menggunakan debat, tetapi tidak mengisolasi pengaruh spesialisasi peran dalam deteksi ambiguitas. Jin et al. (2025) mengusulkan iReDev dengan beberapa agen berbasis peran untuk mendukung pengembangan requirement secara end-to-end Jin et al. (2025). Arsitektur tersebut memperkuat alasan penggunaan peran, tetapi ruang lingkupnya lebih luas daripada task deteksi ambiguitas yang menjadi fokus penelitian ini.

Cheng et al. (2026) menggunakan agen yang terspesialisasi pada atribut kualitas dan negosiasi dialektis dalam RE Cheng et al. (2026). Lu et al. (2026) mengembangkan arsitektur dengan peran analyst, formalizer, dan coder untuk mendukung formalisasi requirement dan code generation Lu et al. (2026). Kedua studi tersebut

mendukung penggunaan spesialisasi, tetapi tidak menyediakan pembandingan langsung role-based versus Self-MoA untuk klasifikasi ambiguitas.

2.2.3 Penerapan Multi-Agent pada Domain Ambigu

Giroh et al. (2025) mengusulkan agen clarifier, planner, dan implementor untuk mentransformasikan Standard Operating Procedure (SOP) yang ambigu menjadi kode Giroh et al. (2025). Walaupun domain SOP berbeda dari requirement perangkat lunak, pola tersebut menunjukkan bahwa pembagian tanggung jawab dapat membantu mengurai sumber ketidakjelasan. Penelitian ini menguji pola pembagian perspektif tersebut pada task yang lebih sempit dan label yang lebih eksplisit.

2.3 Sintesis dan Kesenjangan Penelitian

Tabel 2.2 menyajikan perbandingan ringkas berdasarkan telaah awal yang menjadi dasar draft ini. Nilai “Ya” menunjukkan bahwa aspek tersebut menjadi fokus utama studi, bukan sekadar muncul sebagai komponen pendukung.

Tabel 2.2: Perbandingan penelitian terkait dengan draft penelitian ini

Penelitian	Multi-agent	Role-based	Ambiguitas requirement	Indonesia–Inggris	Vs. Self-MoA
Bashir et al. (2025)	Tidak	Tidak	Ya	Tidak	Tidak
Vijayvargiya et al. (2025)	Ya*	Tidak	Tidak	Tidak	Tidak
Oriol et al. (2025)	Ya	Tidak	Tidak	Tidak	Tidak
Jin et al. (2025)	Ya	Ya	Tidak	Tidak	Tidak
Cheng et al. (2026)	Ya	Ya	Tidak	Tidak	Tidak
Lu et al. (2026)	Ya	Ya	Tidak	Tidak	Tidak
Penelitian ini	Ya	Ya	Ya	Ya	Ya

*: agen interaktif untuk software engineering, bukan fokus utama deteksi ambiguitas requirement.

Kesenjangan yang diuji dalam penelitian ini bukan klaim bahwa belum pernah ada sistem multi-agent untuk RE. Kesenjanganannya lebih spesifik, yaitu belum tersedianya evaluasi terkontrol yang: (1) menjadikan deteksi ambiguitas requirement sebagai task utama, (2) membandingkan spesialisasi peran dengan Self-MoA menggunakan backbone dan anggaran keluaran yang setara, (3) menganalisis kontribusi setiap peran berdasarkan kategori ambiguitas, dan (4) melaporkan perbedaan hasil pada bahasa Indonesia dan Inggris. Validitas kesenjangan tersebut akan ditinjau kembali sebelum proposal final.

2.4 Hipotesis Penelitian

Hipotesis kerja yang akan diuji adalah:

1. H_1 : Role-based multi-agent LLM menghasilkan F1-score deteksi ambiguitas yang lebih tinggi daripada single-agent LLM pada kondisi eksperimen yang sama.
2. H_2 : Role-based multi-agent LLM menghasilkan F1-score yang lebih tinggi atau setara dengan Self-MoA, dengan perbedaan yang dapat ditafsirkan setelah biaya dan interval kepercayaan dilaporkan.
3. H_3 : Pengaruh tiap peran tidak seragam pada seluruh kategori ambiguitas; misalnya, perspektif tester lebih informatif untuk ketidaklengkapan atau kontradiksi yang memengaruhi verifikasi.
4. H_4 : Performa dan biaya inferensi berbeda antara dataset berbahasa Indonesia dan Inggris.

Bab III

METODE DAN RANCANGAN PENELITIAN

3.1 Rancangan Penelitian

Penelitian ini menggunakan rancangan eksperimental komparatif dengan unit analisis berupa satu requirement perangkat lunak. Tujuannya adalah mengukur apakah spesialisasi perspektif pada role-based multi-agent LLM memberikan peningkatan yang dapat dibedakan dari efek pengulangan model yang sama. Alur penelitian terdiri atas persiapan dataset, perancangan arsitektur, pelaksanaan eksperimen, evaluasi, dan analisis validitas.

3.1.1 Tahap 1: Definisi Task dan Preparasi Dataset

Dataset akan dibangun dari dua kelompok bahasa:

1. **Bahasa Inggris:** requirement dari sumber publik yang dapat digunakan untuk penelitian, dengan prioritas pada sumber yang menyediakan konteks dan label ambiguitas atau memungkinkan anotasi ulang.
2. **Bahasa Indonesia:** requirement hasil translasi terkontrol dan/atau requirement berbahasa Indonesia yang tersedia secara terbuka. Setiap item akan diperiksa agar terjemahan tidak menghapus atau menambahkan ambiguitas secara tidak sengaja.

Target awal yang bersifat tentatif adalah sekurang-kurangnya 300 item per bahasa setelah pembersihan. Jumlah final akan ditetapkan setelah audit lisensi, duplikasi, panjang teks, dan distribusi label. Apabila distribusi kategori sangat tidak seimbang, sampling akan dilakukan dengan mempertahankan seluruh item minoritas yang layak dan melaporkan distribusi sebenarnya.

Setiap item akan memiliki metadata minimal berupa identitas sumber, bahasa, teks requirement, label ambiguitas, dan alasan anotasi. Data akan dipisahkan pada tingkat sumber apabila beberapa item berasal dari dokumen atau proyek yang sama. Pemisahan ini mengurangi risiko model menerima pola atau potongan teks yang hampir sama pada tahap pengembangan dan pengujian. Pembagian awal yang akan diuji adalah 60% untuk pengembangan prompt, 20% untuk validasi keputusan desain, dan 20% untuk pengujian akhir.

3.1.2 Tahap 2: Pedoman Anotasi dan Validasi Label

Anotasi menggunakan skema multi-label dengan enam kategori: leksikal, sintaktis, semantik, kekaburan, ketidaklengkapan, dan kontradiksi. Annotator menerima pedoman yang berisi definisi operasional, contoh, kontra-contoh, dan aturan untuk

kasus yang memiliki lebih dari satu label. Selain label, annotator mencatat potongan teks yang menjadi bukti serta alasan singkat.

Prosedur validasi terdiri atas langkah berikut:

1. melakukan pilot annotation pada subset yang mencakup variasi bahasa dan kategori;
2. membandingkan hasil annotator dan memperbaiki definisi yang menimbulkan disagreement berulang;
3. mengukur Inter-Annotator Agreement (IAA) untuk setiap label, menggunakan Cohen's κ apabila terdapat dua annotator atau Krippendorff's α apabila jumlah annotator lebih dari dua;
4. menyelesaikan disagreement melalui diskusi terarah atau annotator adjudicator, sambil menyimpan label awal agar prosesnya dapat diaudit;
5. membekukan label test set sebelum konfigurasi model final dipilih.

3.1.3 Tahap 3: Arsitektur yang Dibandingkan

Semua konfigurasi menggunakan format keluaran terstruktur agar hasil dapat dihitung secara otomatis. Format minimal mencakup keputusan *ambiguous* dan *categories*, serta *field evidence*, *explanation*, dan *confidence*. *Field evidence* wajib merujuk pada bagian teks input; keluaran tanpa bukti tetap disimpan, tetapi dapat ditandai pada analisis *failure mode*.

Tiga konfigurasi utama didefinisikan sebagai berikut:

1. **Single-Agent LLM:** satu model menerima requirement dan prompt analisis umum, kemudian mengeluarkan keputusan akhir. Konfigurasi ini menjadi baseline biaya dan performa.
2. **Self-MoA:** tiga instans model yang sama menerima prompt umum yang sama secara independen. Keluaran ketiganya diteruskan ke aggregator dengan format dan batas token yang sama seperti konfigurasi *role-based*.
3. **Role-Based Multi-Agent LLM:** tiga agen menerima requirement yang sama, tetapi masing-masing memiliki instruksi khusus sebagai *stakeholder*, *developer*, atau *tester*. Aggregator menerima requirement asli dan ketiga analisis untuk menghasilkan keputusan akhir.

Pada konfigurasi *role-based*, pembagian fokus awal ditetapkan sebagai berikut:

- **Stakeholder:** kejelasan kebutuhan bisnis, aktor, tujuan, kondisi penggunaan, dan istilah yang masih samar.

- **Developer:** implementabilitas, interpretasi teknis, dependensi, aturan perilaku, dan struktur kalimat yang dapat menimbulkan makna ganda.
- **Tester:** keterujian, kriteria penerimaan, kondisi batas, ketidaklengkapan, dan konflik yang dapat menyebabkan hasil pengujian berbeda.

Aggregator tidak boleh menambahkan fakta dari luar input. Ia harus memilih label berdasarkan requirement, analisis agen, dan bukti yang dikutip. Aturan ini membantu membedakan peningkatan deteksi dari peningkatan yang hanya berasal dari pengetahuan umum model.

3.1.4 Tahap 4: Protokol Eksperimen

Perbandingan utama menggunakan backbone model yang sama pada ketiga konfigurasi. Variabel penelitian diringkas pada Tabel 3.1.

Tabel 3.1: Variabel dan pengendalian eksperimen

Jenis	Variabel	Cara pengendalian atau pengukuran
Independen	konfigurasi single-agent, Self-MoA, dan role-based multi-agent	prompt dan jumlah peran ditentukan sebelum test run
Independen	bahasa input	blok bahasa Indonesia dan Inggris dianalisis terpisah
Independen	kategori ambiguitas	label multi-label dan metrik per kategori
Kontrol	backbone model, template keluaran, panjang input, dan batas token	model, versi, parameter, dan prompt dicatat di log
Dependen	precision, recall, F1-score, dan exact match	dihitung terhadap label test set
Dependen	biaya dan latensi	token input/output, waktu, dan biaya layanan dicatat per item

Prompt untuk ketiga konfigurasi akan memakai definisi kategori dan format keluaran yang ekuivalen. Prompt berbahasa Indonesia dan Inggris disusun sebagai pasangan semantik, kemudian diuji pada validation set. Setelah dipilih, prompt tidak diubah ketika test set dijalankan. Parameter inferensi utama akan menggunakan nilai deterministik atau nilai terendah yang tersedia pada layanan model; apabila layanan tidak mendukung nilai tersebut, parameter aktual akan dicatat dan diperlakukan sebagai keterbatasan.

Eksperimen dijalankan secara terpisah untuk setiap item dan konfigurasi. Setiap keluaran disimpan bersama model, versi, waktu mulai, waktu selesai, token input, token output, parameter inferensi, prompt identifier, dan status error. Pengulangan terbatas dapat digunakan untuk mengukur variasi keluaran apabila model bersifat nondeterministik.

3.1.5 Tahap 5: Metrik Evaluasi

Untuk setiap kategori ambiguitas, hasil diklasifikasikan sebagai true positive (TP), false positive (FP), false negative (FN), atau true negative (TN). Metrik dasar dihitung sebagai berikut:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

$$F1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Metrik yang dilaporkan meliputi:

- **Metrik klasifikasi:** precision, recall, dan F1-score micro serta macro untuk keseluruhan label.
- **Metrik per kategori:** precision, recall, dan F1-score untuk enam kategori ambiguitas.
- **Metrik per bahasa:** hasil terpisah untuk bahasa Indonesia dan Inggris, termasuk selisih performa antarbahasa.
- **Metrik keputusan item:** akurasi label `ambiguous` dan `exact match` untuk seluruh set kategori yang diprediksi.
- **Metrik reliabilitas:** IAA antarannotator serta persetujuan sistem terhadap label adjudicated.
- **Metrik operasional:** token input/output, latensi, jumlah error, dan biaya inferensi atau estimasi biaya layanan.

Perbedaan performa akan dilengkapi interval kepercayaan 95% melalui paired bootstrap pada item yang sama. Untuk keputusan biner, uji berpasangan seperti McNemar dapat digunakan apabila jumlah item dan asumsi datanya memadai. Nilai p tidak akan menjadi satu-satunya dasar kesimpulan; besar efek, interval kepercayaan, distribusi per kategori, dan biaya juga akan dilaporkan.

3.1.6 Tahap 6: Ablation Study dan Analisis Failure Mode

Ablation study dirancang untuk menjawab apakah hasil berasal dari peran tertentu atau dari proses agregasi secara umum. Ablasi yang direncanakan meliputi:

1. menjalankan setiap peran secara tunggal untuk mengukur kontribusi individual;
2. menghapus satu peran dari arsitektur tiga peran (leave-one-role-out);
3. membandingkan aggregator LLM dengan majority voting yang sederhana;
4. membandingkan Self-MoA dengan jumlah instans satu, dua, dan tiga;
5. menguji pengaruh bukti wajib dan confidence field terhadap precision serta kualitas penjelasan.

Failure mode dikategorikan, antara lain, menjadi false positive, false negative, label kategori yang tertukar, bukti yang tidak mendukung keputusan, konflik antar-agen, dan keluaran yang tidak mengikuti schema. Beberapa kasus paling informatif dari setiap kategori akan dianalisis secara kualitatif untuk menjelaskan mengapa konfigurasi tertentu berhasil atau gagal.

3.2 Ancaman terhadap Validitas

Ancaman terhadap validitas dan mitigasinya adalah sebagai berikut:

1. **Validitas konstruk:** definisi ambiguitas dapat ditafsirkan berbeda oleh annotator. Mitigasi dilakukan melalui pedoman operasional, pilot annotation, IAA, dan pelaporan contoh disagreement.
2. **Validitas internal:** perubahan model, prompt, temperatur, atau batas token dapat menjadi faktor pengganggu. Semua konfigurasi utama menggunakan backbone dan output schema yang sama, sedangkan seluruh parameter dicatat.
3. **Kebocoran data:** item yang sama atau sangat mirip dapat muncul di split berbeda. Mitigasi dilakukan melalui deduplikasi dan pemisahan berdasarkan sumber atau proyek.
4. **Validitas eksternal:** dua bahasa dan sumber publik belum tentu mewakili seluruh praktik RE. Generalisasi dibatasi pada karakteristik dataset yang dilaporkan.
5. **Validitas kesimpulan:** jumlah item, ketidakseimbangan label, dan nondeterminisme model dapat menurunkan daya uji. Interval kepercayaan, pengulangan terbatas, dan analisis per kategori akan dilaporkan bersama hasil utama.

3.3 Alat dan Bahan

Alat dan bahan yang digunakan dalam tahap implementasi awal meliputi:

1. **Model LLM:** satu backbone model yang tersedia melalui Ollama atau API yang kompatibel. Model, versi, parameter, dan tanggal pengambilan hasil dicatat di konfigurasi eksperimen. Model alternatif digunakan hanya sebagai eksperimen tambahan setelah perbandingan utama selesai.
2. **Orkestrator:** Python dengan framework multi-agent yang dipilih, dengan CrewAI sebagai kandidat implementasi awal karena mendukung definisi agent berbasis peran. Logika evaluasi dan agregasi dibuat sebagai modul terpisah agar hasil tidak bergantung pada abstraksi framework.
3. **Pengolahan data:** format JSON atau CSV untuk dataset, keluaran model, metadata eksperimen, dan ringkasan metrik.
4. **Evaluasi:** pustaka Python untuk precision, recall, F1-score, bootstrap, dan pengukuran IAA. Versi pustaka dicatat bersama environment eksperimen.
5. **Infrastruktur:** komputer lokal dan/atau layanan inferensi cloud sesuai keterjangkauan. Eksperimen pilot dilakukan pada subset kecil sebelum pengujian penuh.

3.4 Keluaran Penelitian

Keluaran yang ditargetkan adalah prototipe orkestrasi, pedoman anotasi, dataset atau subset yang dapat dibagikan sesuai lisensi, konfigurasi prompt, log eksperimen, tabel metrik, analisis ablation, dan laporan failure mode. Keluaran yang memuat teks sumber pihak ketiga hanya akan dibagikan apabila izinnnya memungkinkan; metadata dan kode evaluasi tetap dipisahkan agar audit dapat dilakukan tanpa menyalin data berlisensi terbatas.

Bab IV

JADWAL PENELITIAN

4.1 Rencana Kerja

Jadwal berikut merupakan rencana tentatif enam bulan sejak penelitian disetujui. Nomor bulan digunakan agar jadwal tidak mengunci penelitian pada tanggal yang belum disepakati. Setiap perubahan setelah bimbingan akan dicatat pada versi berikutnya.

Kegiatan	B1	B2	B3	B4	B5	B6
Tinjauan pustaka dan pematangan rumusan masalah						
Audit sumber data dan penyusunan pedoman anotasi						
Pilot annotation dan pengukuran IAA						
Implementasi pipeline dan tiga konfigurasi eksperimen						
Pengujian pada validation set dan pembekuan protokol						
Eksperimen test set dan pencatatan log						
Analisis metrik, ablation, dan failure mode						
Penulisan laporan dan konsultasi hasil						

Keterangan: B1–B6 adalah bulan relatif sejak persetujuan rencana penelitian. Blok biru menunjukkan periode aktivitas utama; beberapa aktivitas dapat berjalan paralel apabila dependensinya telah terpenuhi.

DAFTAR PUSTAKA

- Bashir, S., Ferrari, A., Khan, A., Strandberg, P., Haider, Z., Saadatmand, M., and Bohlin, M. (2025). Requirements ambiguity detection and explanation with LLMs: An industrial study. In *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME 2025)*, Malmö. IEEE.
- Berry, D. M. and Kamsties, E. (2004). Ambiguity in requirements specification. In *Proc. 1st Int. Workshop Perspect. Softw. Requirements*, pages 7–12, Auckland.
- Berry, D. M., Kamsties, E., and Krieger, M. M. (2000). From the start: A primer on requirements ambiguity. *ACM SIGSOFT Softw. Eng. Notes*, 25(4):12–13.
- Cheng, H., Kim, M., Khomh, F., Racharak, T., Yoshioka, N., Ubayashi, N., and Washizaki, H. (2026). QUARE: Multi-agent negotiation for balancing quality attributes in requirements engineering. arXiv:2603.11890, diakses 14 Juni 2026.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. arXiv:2305.14325, diakses 14 Juni 2026.
- Ferrara, F., Fattoruso, G., and De Rosa, M. (2014). A natural language inference approach for ambiguity detection in requirements specification. In *Proc. 2014 IEEE Int. Requirements Eng. Conf.*, pages 303–308, Karlskrona. IEEE.
- Giroh, S. K., Ghosh, P., Jain, A., Paunekar, H. G., Rastogi, A., Yenigalla, P., and Nedi-yanchath, A. (2025). Structuring the unstructured: A multi-agent LLM framework for transforming ambiguous SOPs into code. In *Proc. 2025 Conf. Empir. Methods Nat. Lang. Process. Ind. Track*, Miami. ACL.
- Jin, D., Sun, W., Huang, J., Liang, P., Xuan, J., Liu, Y., and Jin, Z. (2025). iReDev: A knowledge-driven multi-agent framework for intelligent requirements development. *ACM Trans. Softw. Eng. Methodol.*
- Kamsties, E., Berry, D. M., and Paech, B. (1998). Detecting ambiguity in requirements: An empirical comparison. *Requirements Engineering*, 3(3):192–209.
- Li, W., Lin, Y., Xia, M., and Jin, C. (2025). Rethinking mixture-of-agents: Is mixing different LLMs beneficial? arXiv:2502.00674, diakses 14 Juni 2026.
- Lu, X., Sun, W., Zhang, Y., Hu, M., Tian, C., Jin, Z., and Liu, Y. (2026). ReDe-Fo: Requirements development and formalization for reliable code generation: A multi-agent vision. arXiv:2508.18675, diakses 14 Juni 2026.
- Oriol, M., Motger, Q., Marco, J., and Franch, X. (2025). Multi-agent debate strategies to enhance requirements engineering with large language models. arXiv:2507.05981, diakses 14 Juni 2026.

- Vijayvargiya, S., Zhou, X., Yerukala, A., Sap, M., and Neubig, G. (2025). Ambig-SWE: Interactive agents to overcome ambiguity in software engineering. arXiv:2502.13069, presented at ICLR 2026, diakses 14 Juni 2026.
- Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., and Zou, J. (2024). Mixture-of-agents enhances large language model capabilities. In *Proc. Int. Conf. Learn. Represent. (ICLR 2025)*, Singapore.
- Wiegers, K. E. (2003). *Software Requirements*. Microsoft Press, Redmond, 2 edition.